IT314 Software Engineering

Bug Reproduction & Triage Assistant

Requirements Engineering Report

Stakeholders, Elicitation, Requirements, Product Backlog, EPICs and Sprint Plan, Conflict Resolution

Group No.: 9

Sr. No.	Name	Student ID
1	Kartik Shah	202401084
2	Dev Parmar	202401138
3	Kevin Rank	202401091
4	Shubham Mishra	202301100
5	Priyanshi Kotadiya	202401097
6	Muneet Sadhwani	202401123
7	Harshil Patel	202401144
8	Yashraj Jadeja	202401072
9	Dhruv Nagani	202401124
10	Katuri Anushree	202401086

Contents

1. Introduction
2. Stakeholders and End Users
3. Requirement Elicitation
4. Requirements
5. Activity Diagram
6. User Stories (Product Backlog)
7. EPICs and Sprint Plan
8. Requirement Conflicts and Resolution

1. Introduction

1.1 Problem

Bug reports written by testers and end users are often a single unstructured paragraph. They regularly leave out the steps to reproduce, the expected and actual behaviour, and the environment in which the bug occurred. Developers then spend time asking follow-up questions, the same bug gets filed more than once, and severity is judged inconsistently.

A typical example of the kind of input we are designing for:

> *"Sometimes, when I enter the wrong password and click Login on Chrome (Windows), the application crashes instead of showing an error message."*

1.2 Proposed System

The Bug Reproduction & Triage Assistant is a web application that takes such a report and converts it into a structured, developer-ready bug report. It extracts reproduction steps, expected and actual behaviour and environment details, points out missing information and asks the reporter for it, suggests severity and priority, searches existing issues for likely duplicates using semantic similarity, and lets the user review and edit everything before exporting it or filing it as a GitHub Issue.

1.3 Scope of this Report

Sections 1 to 5 are the material we submitted for Lab Assignment 6: stakeholders, elicitation techniques and status, the requirements gathered so far, and the activity diagram. Sections 6 to 9 extend that work with the product backlog written as user stories, the grouping of stories into EPICs and sprints, the conflicts we found between requirements and how we resolved them, and the technology decisions taken since. The project follows the Scrum framework, so several requirements are still marked as provisional and will be revised as elicitation continues.

2. Stakeholders and End Users

We identified seven stakeholders. S1 to S3 are the people who use the system or consume its output directly, S4 to S6 use or operate it in a supporting role, and S7 is an external system whose rules constrain our design.

Table 1: Stakeholders and end users

ID	Stakeholder	Type	Interest in the system	Influence
S1	QA testers	Primary end user	Turn rough notes into complete reports quickly and avoid filing duplicates	High
S2	Casual reporters (end users or non-technical members of the product team)	Primary end user	Report a problem in plain language without knowing a bug template	Medium
S3	Software developers	Primary consumer of output	Receive reports that are reproducible and include environment details	High
S4	Project manager / team lead	Secondary user	Consistent severity and priority, fewer duplicates, visibility of bug trends	High
S5	Repository maintainers	Secondary user	Only clean, labelled, non-duplicate issues reach the GitHub repository	Medium
S6	System administrator	Operator	Deploy the system, manage accounts, keep tokens and report data secure	Medium
S7	GitHub (REST API)	External system	Rate limits, token permissions and the API contract constrain the integration	Medium

3. Requirement Elicitation

3.1 Techniques Selected for Each Stakeholder

Table 2: Elicitation techniques and justification

Stakeholder	Technique(s)	Justification
S1 QA testers	Semi-structured interviews, observation	Testers follow habits they rarely describe, such as which fields they skip and how they look for duplicates. Interviews bring out their pain points, and watching them file a bug shows steps they do not mention. We plan to approach seniors and peers who have done QA or development internships.
S2 Casual reporters	Survey, document analysis of real user-written reports	We need a large number of examples of how untrained users describe bugs, and a survey reaches many people cheaply. Public GitHub issues written by end users give us real unstructured text to study and later to test against.
S3 Developers	Interviews, document analysis	Developers can state exactly what makes a report unusable. Closed issues labelled "needs more info" or "cannot reproduce" in open-source repositories show which missing fields matter most.
S4 Project manager	Interview, brainstorming	Few people hold this role, so a focused interview is enough. Brainstorming helps us define the rules separating severity from priority and decide which analytics are useful.
S5 Repository maintainers	Document analysis (issue templates, CONTRIBUTING files, label schemes), short survey	Maintainers have already written most of their expectations into issue templates and contribution guidelines.
S6 System administrator	Brainstorming against a security checklist	Deployment, access control and data protection needs are best derived by reviewing a standard security checklist within the group.
S7 GitHub API	Document analysis (GitHub REST API documentation)	The rules of the API are fully documented and cannot be gathered any other way.

3.2 Elicitation Instruments

Interview guide (S1, S3, S4), about 20 minutes

1. Walk us through the last bug report you wrote or received.
2. What information is most often missing when you receive a bug report?
3. How do you check whether a bug has already been reported?
4. How do you decide severity, and who decides priority?
5. Would you trust a tool that suggests severity? What would make you trust it?
6. What would stop you from using a tool like this?

Survey (S2, S5), about 5 minutes

1. Describe the last software problem you reported, in your own words. (Responses also serve as test data.)
2. Do you usually know your browser version or app version? (Yes / No / Don't know how to check)
3. On a scale of 1 to 5, how willing are you to answer follow-up questions from a tool?
4. Maintainers only: which labels and template fields do you require on new issues?

Observation checklist (S1)

- Time taken to write a complete report from scratch.
- Fields the tester fills in without being prompted, and fields left out.

- Whether and how the tester searches for existing issues before filing.

3.3 Current Elicitation Status

Document analysis is complete for the sources available to us. Interviews, surveys and observation have not been conducted yet, so the requirements that depend on them are marked as provisional in Section 4.

Table 3: Elicitation status per stakeholder

Stakeholder	Technique	Status	Gathered so far / next action
S7 GitHub API	Document analysis	Complete	Issue creation fields, token permissions and rate limits recorded (DR-08, DR-09).
S3 Developers	Document analysis	Complete	Common missing fields identified from open-source issues. Next: 3 to 5 interviews.
S5 Maintainers	Document analysis	Complete	Template fields and label conventions recorded. Survey is optional.
S3 Developers	Interviews	Not started	Interview guide ready; participants to be contacted.
S1 QA testers	Interviews, observation	Not started	Recruit 3 to 5 participants; guide and checklist ready.
S2 Casual reporters	Survey	Not started	Form to be circulated; target of at least 30 responses.
S4 Project manager	Interview, brainstorming	In progress	Initial severity and priority rules drafted through brainstorming. Interview pending.
S6 System administrator	Brainstorming	In progress	Security requirements drafted (NFR-03 to NFR-06). Checklist review pending.

4. Requirements

Each requirement has an ID, a MoSCoW priority (M = Must, S = Should, C = Could), the stakeholder it came from, and a status. Confirmed means the requirement is stated in the assigned problem statement. Provisional means it came from document analysis or brainstorming and still has to be validated through interviews or surveys. We chose this structure so that every requirement can be traced back to its source and re-examined when elicitation results come in.

4.1 Functional Requirements

Table 4: Functional requirements

ID	Requirement	Pri	Source	Status
FR-01	Users can register, log in and log out.	M	S6	Provisional
FR-02	Users can view, edit and delete only their own bug reports.	M	S6	Provisional
FR-03	Users can submit a free-text bug report of 20 to 10,000 characters.	M	S1, S2	Confirmed
FR-04	Users can optionally paste logs or a stack trace in a separate field.	S	S3	Provisional
FR-05	The system extracts a title, summary, expected behaviour and actual behaviour from the report text.	M	Brief	Confirmed
FR-06	The system extracts reproduction steps as an ordered list, including preconditions.	M	Brief	Confirmed
FR-07	The system extracts every environment detail mentioned in the report as a name and value pair (for example browser, operating system, runtime version, database, deployment), rather than a fixed list of fields.	M	Brief	Confirmed
FR-08	The system extracts error messages and stack-trace lines.	S	Brief, S3	Confirmed
FR-09	Every extracted field is labelled as Explicit, Inferred, Unverified or Missing, based on the verification described in FR-27.	M	S3	Provisional

Table 4 (continued)

ID	Requirement	Pri	Source	Status
FR-10	The system shows the part of the original text each extracted field came from.	S	S3	Provisional
FR-11	The system lists missing required fields and shows a completeness score.	M	Brief	Confirmed
FR-12	The system generates one follow-up question per missing field and merges the user's answers into the report.	M	Brief	Confirmed
FR-13	The system suggests a severity (Critical, High, Medium, Low) along with the reason for it.	M	Brief, S4	Confirmed
FR-14	The system suggests priority, category and reproducibility.	S	Brief, S4	Confirmed
FR-15	Users can override every triage suggestion, and overrides are recorded.	M	S4	Provisional
FR-16	Existing issues can be imported (CSV or JSON) into the issue store used for duplicate search.	M	Brief	Provisional
FR-17	The system ranks the five most similar existing issues by semantic similarity and shows a score and a label (Likely or Possible duplicate).	M	Brief	Confirmed
FR-18	Users can mark a candidate as a confirmed duplicate or as not a duplicate.	S	S1, S5	Provisional
FR-19	The system generates a structured, developer-ready report in a fixed template.	M	Brief	Confirmed
FR-20	Users can edit every field of the generated report and reset it to the suggested version.	M	Brief	Confirmed
FR-21	Users can copy or export the report as Markdown.	S	S3	Provisional
FR-22	Users can connect a GitHub account or token and select a repository.	S	Brief, S7	Confirmed
FR-23	Users can preview an issue and create it on GitHub only after explicit confirmation; the issue URL is stored.	S	Brief, S5	Confirmed
FR-24	Users can view their report history with search and filters (severity, status, date).	M	S1, S4	Provisional
FR-25	A dashboard shows report counts by severity, category and time, and the most common missing fields.	S	S4	Provisional
FR-26	Administrators can deactivate users and configure the duplicate thresholds.	C	S6	Provisional

The functional requirements below were added after the Lab 6 submission, when the extraction approach changed from rule-based parsing to language-model extraction with deterministic verification in Python. Section 9.2 records that change and the reasoning behind it.

Table 5: Functional requirements added after the Lab 6 submission

ID	Requirement	Pri	Source	Status
FR-27	For every extracted field the system verifies that the quoted source text appears verbatim in the report, and that the quoted text supports the extracted value. Fields that fail the first check are labelled Unverified and those that fail the second are labelled Inferred. No report is rejected and no value is silently discarded.	M	S3, S9	Provisional
FR-28	Model output is validated against a fixed schema. On failure the system retries once with the validation error, and falls back to an empty analysis with all fields Missing rather than failing the request.	M	S9	Provisional
FR-29	Before analysis the user can review what the system masked, restore a false positive, and mask any further text manually. All occurrences of a manually masked value are masked.	S	S6, S2	Provisional
FR-30	The system treats report text as data and not as instruction. Instructions embedded in a report must not change the analysis, and this is checked by an automated test suite.	M	S6	Provisional

Table 5 (continued)

ID	Requirement	Pri	Source	Status
FR-31	After analysis the user is shown the environment details that were extracted, can correct them, and is asked whether any further detail is needed to reproduce the bug.	M	S1, S3	Provisional
FR-32	Follow-up questions are proposed by the model with a stated reason and are then filtered in code: questions for fields that are already filled are removed, questions for missing essential fields are always included, and the total is capped.	M	S2	Provisional
FR-33	Every extracted field carries its own source quote, confidence and reason, which are shown to the user.	S	S3	Provisional

4.2 Non-Functional Requirements

Table 6: Non-functional requirements

ID	Category	Requirement (measurable)	Pri	Status
NFR-01	Performance	Analysis of a report of up to 2,000 characters completes in under 5 seconds for 95% of requests on the demo machine.	M	Provisional
NFR-02	Performance	Duplicate search over 100 stored issues returns results in under 2 seconds.	M	Provisional
NFR-03	Security	Passwords are stored only as salted hashes (bcrypt or argon2); login sessions expire after at most 60 minutes of validity.	M	Provisional
NFR-04	Security	GitHub tokens are encrypted at rest and are never returned to the browser or written to logs.	M	Provisional
NFR-05	Privacy	Emails, passwords, API keys and tokens detected in report text are masked before storage and before being sent to any model.	S	Provisional
NFR-06	Reliability	If GitHub is unavailable, all other features keep working and the draft issue is not lost.	M	Confirmed
NFR-07	Usability	A first-time user can submit and review a report without assistance within 3 minutes, checked with at least 5 test users.	S	Provisional
NFR-08	Explainability	Every triage suggestion and duplicate candidate is shown with the reason behind it.	M	Provisional
NFR-09	Accuracy	The system should accurately extract bug information, generate structured reports, and identify similar existing issues using semantic similarity.	S	Provisional
NFR-10	Maintainability	The system should have modular and well-documented code so that developers can easily update the extraction, duplicate detection, and issue generation modules.	S	Provisional
NFR-11	Portability	The system should be deployable on different operating systems using containerization, with minimal configuration changes.	S	Provisional
NFR-12	Availability	The system should remain usable when external services such as the GitHub API are temporarily unavailable, allowing users to save or export bug reports locally.	M	Provisional

4.3 Domain Requirements

Domain requirements come from established practice in software testing and issue tracking, and apply regardless of how our particular features are implemented.

Table 8: Domain requirements

ID	Requirement	Source
DR-01	A bug report must contain at least a summary, steps to reproduce, the expected result, the actual result and the environment. This matches standard defect report content (ISTQB) and common issue templates.	Document analysis

Table 8 (continued)

ID	Requirement	Source
DR-02	Severity (technical impact) and priority (urgency of the fix) are separate attributes and must not be merged into one field.	Document analysis, S4
DR-03	Severity scale: Critical (crash, data loss or security breach with no workaround), High (major function broken), Medium (function impaired but a workaround exists), Low (cosmetic or minor).	Document analysis
DR-04	Reproducibility is recorded as Always, Sometimes, Rarely, Unable to reproduce, or Unknown.	Document analysis
DR-05	Only a person can confirm that two reports are duplicates. A similarity score is supporting evidence, not a decision.	S3, S5
DR-06	Reproduction steps are ordered, single actions written in the imperative form (for example, "Click Login").	S3
DR-07	Environment values must not be assumed. An unknown version remains "Not provided".	S3
DR-08	A GitHub issue consists of a title, a Markdown body, and optional labels and assignees. Creating one requires a token with issue write access to that repository.	S7
DR-09	GitHub API rate limits apply (5,000 requests per hour for authenticated requests), so 403 and 429 responses must be handled.	S7
DR-10	A report moves through a basic lifecycle: Draft, Analysed, Reviewed, Exported or Submitted.	S4

5. User Stories (Product Backlog)

Each story is written as a card. The front of the card gives the role, goal and benefit; the back gives the conditions of satisfaction that must hold for the story to be accepted. Story points follow the Fibonacci scale and priorities follow MoSCoW.

EPIC 1: Access & Accounts

Table 9: User stories for Access & Accounts

ID	Front of card	Back of card (conditions of satisfaction)	Pts	Pri	Req.
US-01	As a QA tester, I want to register and log in, so that my reports stay private to me.	1. Registration requires a valid email and a password of at least 8 characters. 2. A duplicate email shows "Account already exists". 3. Wrong credentials show a generic error that does not reveal which field was wrong. 4. Passwords are stored hashed. 5. Logging out ends the session.	3	M	FR-01
US-02	As a user, I want only my own reports to be visible to me, so that nobody else can read my data.	1. The report list returns only the logged-in user's reports. 2. Requesting another user's report ID returns 404 (NFR-15). 3. An automated test covers both cases.	2	M	FR-02
US-03	As an administrator, I want to deactivate accounts, so that access can be revoked.	1. A deactivated user cannot log in. 2. The user's data remains intact. 3. Only the administrator role sees this option.	3	C	FR-26

EPIC 2: Report Submission & History

Table 10: User stories for Report Submission & History

ID	Front of card	Back of card (conditions of satisfaction)	Pts	Pri	Req.
US-04	As a QA tester, I want to paste an unstructured bug description and submit it, so that the system can structure it for me.	1. The input box accepts 20 to 10,000 characters. 2. Empty or too-short input shows an inline error and is not submitted. 3. The original text is saved unchanged with a timestamp. 4. A Clear button empties the box after confirmation.	3	M	FR-03
US-05	As a developer, I want reporters to paste logs separately, so that stack traces are not mixed into the description.	1. An optional log field accepts up to 50,000 characters. 2. Logs are displayed in a monospace font, unmodified.	2	S	FR-04
US-06	As a QA tester, I want to see my past reports with search and filters, so that I can find earlier bugs quickly.	1. The list is sorted newest first and shows title, severity, status and date. 2. Reports can be filtered by severity and status. 3. Keyword search covers the title and original text. 4. An empty state explains how to create the first report.	5	M	FR-24
US-07	As a QA tester, I want to edit or delete a report, so that I can correct mistakes.	1. After editing the original text, analysis can be run again. 2. Deletion asks for confirmation and removes all linked analysis data.	3	S	FR-02
US-32	As a reporter, I want to see and adjust what the system masked before my report is analysed, so that nothing private is sent without my knowledge.	1. Detected items are listed with their type before analysis begins. 2. Any detection can be restored if it is a false positive. 3. Any selected text can be masked manually, and every occurrence of that text is masked. 4. When nothing is detected, a short link still allows a review. 5. Only the masked copy is sent onward; the original remains visible to the owner.	5	S	FR-29

EPIC 3: Information Extraction & Reproduction Analysis

Table 11: User stories for Information Extraction & Reproduction Analysis

ID	Front of card	Back of card (conditions of satisfaction)	Pts	Pri	Req.
US-08	As a developer, I want the title, summary, expected behaviour and actual behaviour extracted, so that I understand the bug without rereading the paragraph.	1. For the sample report in Section 1.1, expected behaviour is “an error message is shown” and actual behaviour is “the application closes”. 2. Each field is labelled Explicit, Inferred or Missing. 3. A field that cannot be found is shown as Missing and is never invented. 4. Every field carries the quoted text it came from, and the label follows the verification in FR-27. 5. Analysis completes within the limit in NFR-19.	8	M	FR-05, FR-09, FR-27

Table 11 (continued)

ID	Front of card	Back of card (conditions of satisfaction)	Pts	Pri	Req.
US-09	As a developer, I want the reproduction steps as a numbered list, so that I can follow them exactly.	1. Actions appear in the same order as in the original text. 2. Conditions such as "wrong password" are captured as a step or precondition. 3. If no actions are found, the steps field is marked Missing. 4. Each step carries the quoted text it came from. Because steps are condensed rather than copied, a step counts as Explicit when the quote shares its main action and object, and as Inferred otherwise. 5. A report producing more than seven steps is flagged for human review rather than shortened automatically.	8	M	FR-06, FR-27
US-10	As a developer, I want every environment detail the reporter mentioned to be captured, so that I can set up the same conditions whatever the technology involved.	1. Each detail is stored as a name and value pair with the quoted text it came from, for example browser = Chrome, runtime = Python 3.11, database = PostgreSQL 14. 2. No fixed list of environment fields is assumed, so reports about web, backend, mobile or infrastructure are all handled the same way. 3. Names are normalised so that the same concept written differently is stored under one name. 4. A value that cannot be traced to the report text is labelled Inferred or Unverified and never presented as stated fact.	5	M	FR-07, FR-09
US-11	As a developer, I want error messages and stack-trace lines highlighted, so that I can search the codebase for them.	1. Quoted error strings and exception names are extracted. 2. Stack-trace lines from the log field are listed separately.	3	S	FR-08
US-12	As a developer, I want to see which part of the text each field came from, so that I can verify the extraction.	1. Selecting a field highlights its source text in the original report. 2. Inferred fields show the rule that produced them.	3	S	FR-10

EPIC 4: Missing Information & Follow-up

Table 12: User stories for Missing Information & Follow-up

ID	Front of card	Back of card (conditions of satisfaction)	Pts	Pri	Req.
US-13	As a QA tester, I want to see what is still missing from my report, so that I know what to add before filing.	1. The essential fields are expected behaviour, actual behaviour, reproduction steps, frequency and at least one environment detail. 2. Each missing essential field is flagged. 3. A completeness score is shown, counting only fields that are present and verified. 4. Fields labelled Inferred or Unverified are shown as needing confirmation rather than counted as present.	3	M	FR-11

Table 12 (continued)

ID	Front of card	Back of card (conditions of satisfaction)	Pts	Pri	Req.
US-14	As a casual reporter, I want to be asked about what is missing in plain language, so that I can complete the report without knowing a template.	1. Questions are proposed by the model with a stated reason, so that details specific to this bug can be asked about. 2. Questions for fields that are already present and verified are removed automatically. 3. A question is always included for any missing essential field, even when the model does not propose one. 4. No more than six questions are shown at once. 5. Answers are merged into the report as stated by the user, and the completeness score updates immediately. 6. Questions can be skipped; the report can still be exported with a warning.	5	M	FR-12, FR-32
US-33	As a reporter, I want to check and correct the environment details that were captured and add anything else that matters, so that the developer is not missing something only I would know.	1. The extracted environment details are shown with their source text and can be corrected or removed. 2. One open question asks whether anything else about the setup, account, data or timing is relevant. 3. When nothing was extracted, a specific prompt asks what the reporter was using. 4. The answer is passed through the same extraction and verification path and stored as structured entries.	5	M	FR-31

EPIC 5: Triage Assistance

Table 13: User stories for Triage Assistance

ID	Front of card	Back of card (conditions of satisfaction)	Pts	Pri	Req.
US-15	As a project manager, I want a suggested severity with a reason, so that triage is faster and more consistent.	1. The suggestion is one of Critical, High, Medium or Low. 2. The reason names what triggered it, for example “mentions a crash, so High”. 3. The value is shown in an editable dropdown. 4. If the user changes it, the override is saved and flagged.	5	M	FR-13, FR-15
US-16	As a project manager, I want suggested priority kept separate from severity, so that business urgency is decided on its own.	1. Priority is a separate field (DR-02). 2. It is labelled as a suggestion for the project manager to decide. 3. It is editable.	3	S	FR-14
US-17	As a developer, I want category and reproducibility suggested, so that the issue reaches the right person.	1. Category comes from a fixed list: UI, Authentication, Performance, Database, API, Functional, Security, Compatibility. 2. “sometimes” in the text gives Reproducibility = Sometimes. 3. With no signal in the text, the value is Unknown.	3	S	FR-14
US-18	As a project manager, I want to see who changed a triage value, so that decisions can be traced.	1. A change log shows the user, old value, new value and time.	2	C	FR-15

EPIC 6: Duplicate Detection

Table 14: User stories for Duplicate Detection

ID	Front of card	Back of card (conditions of satisfaction)	Pts	Pri	Req.
US-19	As an administrator, I want to import existing issues, so that new reports can be compared against them.	1. CSV or JSON files with id, title, body and url are accepted. 2. Invalid rows are reported and skipped. 3. Similarity representations are computed at import time.	5	M	FR-16
US-20	As a QA tester, I want to see the most similar existing issues, so that I do not file a duplicate.	1. The top five candidates are shown, sorted by similarity score. 2. A score of 0.80 or above is labelled "Likely duplicate", 0.60 to 0.80 "Possible duplicate", and lower scores are hidden. These starting thresholds will be tuned. 3. The wording never states that a report is a duplicate. 4. Results are returned within the limit in NFR-02.	8	M	FR-17
US-21	As a repository maintainer, I want testers to confirm or reject candidates, so that we collect real duplicate data.	1. Each candidate has "Duplicate" and "Not duplicate" buttons. 2. The choice is stored and shown on the report.	3	S	FR-18
US-22	As a developer, I want to see why an issue was considered similar, so that I can judge it quickly.	1. Shared key terms and matching fields (for example, same component or same error) are shown.	3	C	FR-17

EPIC 7: Issue Generation & Editing

Table 15: User stories for Issue Generation & Editing

ID	Front of card	Back of card (conditions of satisfaction)	Pts	Pri	Req.
US-23	As a developer, I want a clean report in a standard template, so that every bug reads the same way.	1. Sections: Title, Summary, Steps to Reproduce, Expected, Actual, Environment, Error Details, Severity, Priority, Category, Reproducibility, Possible Duplicates, Additional Information. 2. Missing values appear as "Not provided". 3. Duplicate candidates appear as links.	5	M	FR-19
US-24	As a QA tester, I want to edit every field of the generated report, so that the final version is my own.	1. All fields are editable. 2. Saving persists the changes. 3. "Reset to suggested" restores the system version.	5	M	FR-20
US-25	As a QA tester, I want to copy the report as Markdown, so that I can paste it into any tracker.	1. The report is copied with one click. 2. The Markdown renders correctly on GitHub.	2	S	FR-21

EPIC 8: GitHub Integration

Table 16: User stories for GitHub Integration

ID	Front of card	Back of card (conditions of satisfaction)	Pts	Pri	Req.
US-26	As a QA tester, I want to connect GitHub and choose a repository, so that I can file issues directly.	1. A fine-grained access token is accepted. 2. An invalid token shows a clear error. 3. The token is encrypted at rest and never sent back to the browser. 4. Only repositories where the token can create issues are listed.	5	S	FR-22

Table 16 (continued)

ID	Front of card	Back of card (conditions of satisfaction)	Pts	Pri	Req.
US-27	As a repository maintainer, I want issues created only after the reporter previews and confirms them, so that the repository stays clean.	1. The preview shows the exact title, body and labels. 2. Nothing is sent until Confirm is clicked. 3. If a Likely duplicate exists, the user must acknowledge it first. 4. Clicking Confirm twice does not create two issues. 5. The issue URL is stored. 6. On failure, the draft is kept and the error is shown.	5	S	FR-23
US-28	As a repository maintainer, I want the repository's open issues imported automatically, so that duplicate search uses current data.	1. Open issues are fetched through the API with pagination. 2. Rate-limit responses are handled without losing progress.	3	C	FR-16

EPIC 9: Analytics

Table 17: User stories for Analytics

ID	Front of card	Back of card (conditions of satisfaction)	Pts	Pri	Req.
US-29	As a project manager, I want a dashboard of bugs by severity, category and time, so that I can spot trends.	1. Three charts are built from the user's own reports. 2. An empty state is shown when there is no data.	5	S	FR-25
US-30	As a project manager, I want to see the most commonly missing fields, so that we can guide reporters.	1. A ranked list with counts is shown.	3	C	FR-25
US-31	As an administrator, I want to configure the duplicate thresholds, so that the system can be tuned without code changes.	1. Two numeric thresholds between 0 and 1 can be set, with validation.	2	C	FR-26

Backlog size: Must = 70 points, Should = 37 points, Could = 16 points, for a total of 123 points across 31 stories.

6. EPICs and Sprint Plan

6.1 EPIC Structure

We started from ten candidate EPICs and reduced them to nine. "Bug Reproduction Analysis" and "AI Information Extraction" were merged into EPIC 3, because the steps, expected and actual behaviour, and environment details all come out of the same extraction pipeline; keeping them apart would have made every story in one EPIC depend on the other. "Bug History" was moved into EPIC 2, since history is the list view of submitted reports.

Table 18: EPIC summary

EPIC	Name	Stories	Points
E1	Access & Accounts	US-01 to US-03	8
E2	Report Submission & History	US-04 to US-07, US-32	18
E3	Information Extraction & Reproduction Analysis	US-08 to US-12	27
E4	Missing Information & Follow-up	US-13, US-14, US-33	13
E5	Triage Assistance	US-15 to US-18	13

Table 18 (continued)

EPIC	Name	Stories	Points
E6	Duplicate Detection	US-19 to US-22	19
E7	Issue Generation & Editing	US-23 to US-25	12
E8	GitHub Integration	US-26 to US-28	13
E9	Analytics	US-29 to US-31	10

6.2 Sprint Plan

With roughly one to two months until final submission and a team of ten, we plan four development sprints of about 1.5 to 2 weeks each, followed by a short fifth sprint for hardening and the final demonstration. We expect a velocity of around 25 to 35 points per sprint, since all members are working on this alongside other courses.

Table 19: Sprint plan

Sprint	Sprint goal	Stories	Pts	Other tasks
Sprint 1: Foundation	A user can log in, submit a report and see it in their history.	US-01, 02, 04, 05, 06, 07	18	Repository and branch setup, Slack channels Technology stack decision Database schema and local container setup Concept poster Run interviews and circulate survey
Sprint 2: Extraction and Verification	A submitted report is converted into verified structured fields, with the gaps turned into questions.	US-08 to 14, US-32, US33	45	Define the extraction schema and write the system prompt with its guardrails Build the verification layer and its test suite Build the masking and question-filtering logic Write the prompt injection test suite Revise provisional requirements using elicitation results
Sprint 3: Triage, Duplicates, Report	Suggest triage values, find likely duplicates and produce the clean report.	US-15, 16, 17, 19, 20, 23	29	Import a corpus of existing issues (for example, public GitHub issues) Set the initial duplicate thresholds and record them as an estimate to be revisited
Sprint 4: Editing, GitHub, Analytics	Finalise the reviewable output and the integrations.	US-21, 24, 25, 26, 27, 29	25	Usability test
Sprint 5: Hardening	Stabilise and prepare the final demonstration.	Could-have stories only if time permits (US-03, 18, 22, 28, 30, 31)	–	Security testing Bug fixing, documentation, deployment

Sprint 2 carries the most points because extraction and its verification are the core that every later feature depends on. If velocity falls short, US-12 and US-32 are the first stories to move to Sprint 3. US-23 (report generation) is placed in Sprint 3 rather than Sprint 4, because the product cannot be demonstrated end to end without it and it should not compete for time with the GitHub work.

6.3 MVP Definition

The minimum viable product consists of all Must stories (70 points): accounts, report submission and history, extraction, missing-information detection with follow-up questions, severity suggestion, duplicate detection, and generation and editing of the clean report. GitHub integration is a Should item; we intend to build it in Sprint 4, but the product is complete and usable without it.

7. Requirement Conflicts and Resolution

While cross-checking the requirements and user stories, we found the following conflicts between stakeholders. Each was discussed within the group and resolved as shown below.

Table 20: Requirement conflicts and resolutions

ID	Conflict and stakeholders	Why it conflicts	Resolution	Impact
C1	Filling gaps automatically vs. never inventing information. S1 and S2 want gaps filled; S3 wants only facts (DR-07).	Filling a gap means guessing, and a wrong OS or version sends the developer in the wrong direction.	Three-state labels (FR-09). Inferred values are visually distinct and exported as "Inferred" until the user confirms them. Missing fields become follow-up questions.	Labelling added to US-08 and US-10.
C2	Speed of filing vs. a clean repository. S1 and S4 want one-click filing; S5 wants no noise or duplicates.	Automatic submission removes the human check.	Issues are never submitted automatically. Preview and confirmation are mandatory, and a Likely duplicate must be acknowledged first (US-27).	One extra step for the user; the repository stays clean.
C3	Who decides priority. The system is expected to suggest it; S4 considers priority a business decision.	The system can judge technical impact but not business urgency.	Severity is suggested with a reason. Priority is shown as a suggestion for the project manager, and every override is logged (US-15, 16, 18).	System stays advisory; DR-02 is satisfied.
C4	Duplicate recall vs. noise and response time. S5 wants no duplicate missed; S1 wants a short, relevant list; NFR-02 requires fast results.	Lower thresholds catch more duplicates but flood the list; heavier models are slower.	Show only the top five, use two thresholds and a compact embedding model, and add a dedicated vector store only if the corpus grows beyond about 10,000 issues.	The thresholds start as a documented estimate and are revised as real reports come in.
C5	Keeping full reports vs. privacy. S4 wants history and analytics; S6 and NFR-06 want minimal sensitive data.	Pasted reports often contain emails, passwords or tokens.	Detected secrets are masked before storage and before any model call, analytics use aggregates only, and users can delete a report along with all derived data (US-07).	Small risk that masking hides a string relevant to the bug.
C6	Model coverage vs. verifiable output. S1 and S2 write reports in unpredictable language that only a language model handles well; S3 needs output that can be checked; S6 does not want report data leaving the system unnecessarily.	A model reads any phrasing but can state something the report never said, and its own explanation of its answer is generated text rather than evidence.	The model extracts, but every field must cite the text it came from, and code checks that the quote exists in the report and supports the value (FR-27). What fails the check is labelled rather than trusted, and only masked text is sent.	Coverage of unusual reports without treating model output as fact; the checking code becomes the core engineering work.
C7	Minimal input vs. complete reports. S2 wants to type a single sentence; S3 wants every field filled.	Making many fields mandatory drives casual reporters away.	Five essential fields only, and at most six questions at a time. Questions can be skipped, and export is allowed with a visible "incomplete" warning instead of being blocked.	Some exported reports remain incomplete, but they are clearly marked.
C8	Feature breadth vs. available time. S4 wants analytics and admin tools; S1 and S3 need the core workflow to work reliably within one to two months.	Everything cannot be built to a good standard in the time available.	MoSCoW prioritisation: analytics and GitHub are Should, administration features are Could, and ideas such as screenshot analysis or Jira support are kept out of scope.	A smaller but complete MVP.
C9	Convenient GitHub access vs. token security. S1 wants to connect once; S6 does not want stored credentials.	A stored token is a leak risk.	Fine-grained token limited to issue access, encrypted at rest, never returned to the browser or logged, and removable by the user (US-26).	Slightly more setup for the user.